AI Engineering

Building Applications with Foundation Models

Chip Huyen

Data Quality

A small amount of high-quality data can outperform a large amount of noisy data, e.g., data that is irrelevant or inconsistent. The creators of the Yi model family found that 10K carefully crafted instructions are superior to hundreds of thousands of noisy instructions (Young et al., 2024).

Similarly, "LIMA: Less Is More for Alignment" (Zhou et al., 2023) shows that a 65B-parameter Llama model, finetuned with 1,000 carefully curated prompts and responses, can produce answers that are either equivalent or strictly preferred to GPT-4 in 43% of cases, as judged by human annotators. However, the downside of having too few data examples is that LIMA is not as robust as product-grade models.

The Llama 3 team also arrived at the same conclusion. Notably, they found that human-generated data is more prone to errors and inconsistencies, particularly for nuanced safety policies. This led them to develop AI-assisted annotation tools to ensure high data quality.

Most people understand the importance of data quality, but what does it mean for data to be high-quality? The short answer is that data is considered high-quality if it helps you do your job efficiently and reliably. The long answers, however, differ for different people.[3] In general, data can be considered high-quality if it has the following six characteristics: relevant, aligned with task requirements, consistent, correctly formatted, unique, and compliant. Some specific use cases might have other requirements:

Relevant

The training examples should be relevant to the task you're training the model to do. For example, if the task is to answer legal questions today, a legal dataset from the 19th century might not be relevant. However, if the task is about the legal system in the 19th century, this dataset is highly relevant.

Aligned with task requirements

The annotations should align with the task's requirements. For example, if the task requires factual consistency, the annotations should be factually correct. If the task requires creativity, the annotations should be creative. If the task demands not just a score but also a justification for that score, the annotations should include both scores and justifications. But if the task demands concise answers, the annotations should be concise.

I used "aligned" instead of "accurate" or "correct" because, depending on the task, an accurate or correct response might not be what a user wants.

Consistent

Annotations should be consistent across examples and annotators. If you ask two annotators to annotate the same example, their annotations shouldn't be too different. If the task is to score essays from 1 to 5, would two essays with the same score be of the same quality? Inconsistent annotations can confuse the model, making it harder for the model to learn.

Having a good annotation guideline is essential for having annotations that are both aligned with task requirements and consistent.

Correctly formatted

All examples should follow the format expected by the model. Redundant formatting tokens can interfere with the model's learning, and, therefore, they should be removed. For example, if you scrape product reviews from a website, you should remove HTML tags. Beware of trailing white spaces, new lines, inconsistent casing, and numerical formats.[4]

Sufficiently unique

This refers to unique examples in your data.[5] In the context of model training, duplications can introduce biases and cause data contamination. I use "sufficiently unique" because specific use cases can tolerate different levels of duplications.

Compliant

Data should be compliant with all relevant internal and external policies (including laws and regulations). For example, if you're not allowed to use PII data to train your models, your data shouldn't contain any PII data.

Before setting out to create data, it's important to think about what each of these characteristics means for you. The techniques discussed in this section aim to produce data with these characteristics.

Data Coverage

A model's training data should cover the range of problems you expect it to solve. Real-world users often have a wide range of problems, and the way they express those problems can vary significantly. Having data that captures the diverse usage patterns of your application is key for the model to perform well. Coverage requires sufficient *data diversity*, which is why many refer to this attribute as data diversity.

For example, if some users construct detailed instructions with abundant references while some other users prefer short instructions, your finetuning data should include both detailed and short instructions. If user queries typically have typos, you should include examples with typos. If your application works with multiple programming languages, your training data should include the programming languages your users care about.